HW1 for CS 6210

You may (and probably should) talk about problems with the each other, with the TA, and with me, providing attribution for any good ideas you might get. Your final write-up should be your own.

```
1 using LinearAlgebra
```

1. All about you

- How do you prefer to be called?

Xinyi (sounds like Shin-ee)

- Why are you taking the class?

Want to deepen understanding of numerical linear algebra and learning basic concepts for computational science.

- Are there things you particularly hope to learn?

efficient algorithms for large-scale problems, error analysis, and new practical programming implementations in Julia.

- Do you have any concerns (about background, schedule, etc)?

No major concerns so far, but need time to familiar with Julia as this is the first time to learn this programming languages.

- Is there anything else I should know about your situation?

I want to apply for PhD and want to learn more to connect theory to practice with numerical computations.

2. A little differentiation

Differentiate $\|Ax\|^2$ with respect to x and A, and write the result in the form

$$\delta[\|Ax\|^2] = g^T \delta x + \langle G, \delta A \rangle_F$$

You may use the following tester as a sanity check.

Differentiation: Using Frobenius inner product $\langle G, \delta A \rangle_F = \mathrm{tr}(G^T \delta A)$.
$\|Ax\|^2 = (Ax)^T (Ax) = x^T A^T A x$.

When treat A as constant to differentiate with respect to x, considering a perturbation $\boldsymbol{x} \to \boldsymbol{x} + \boldsymbol{\delta x}$: $\boldsymbol{\delta[\|Ax\|^2] = \delta(x^T A^T Ax) = (\delta x)^T A^T Ax + x^T A^T A(\delta x)}$. Since $\boldsymbol{A^T A}$ is symmetric, this can simplify to $\boldsymbol{2x^T A^T A \delta x = (2A^T Ax)^T \delta x}$ and it identifies $\boldsymbol{g = 2A^T Ax}$.

Then, differentiate with respect to A, considering a pertubation $\boldsymbol{A \to A + \delta A}$: Think $\boldsymbol{\|Ax\|^2 = \sum_{i=1}^{m} [(Ax)_i]^2}$, and then $\boldsymbol{\frac{\partial}{\partial a_{ij}} \|Ax\|^2 = 2(Ax)_i x_j}$. THus, $\boldsymbol{g_{ij} = 2(Ax)_i x_j}$ and $\boldsymbol{G = 2(Ax)x^T}$.

Therefore, $\boldsymbol{\delta[\|Ax\|^2] = (2A^T Ax)^T \delta x + \langle 2(Ax)x^T, \delta A \rangle_F}$, the Frobenius term is $\boldsymbol{\langle G, \delta A \rangle_F = \mathrm{tr}(G^T \delta A) = \mathrm{tr}(2x(Ax)^T \delta A) = 2(Ax)^T (\delta A x)}$.

```
check_p2 (generic function with 2 methods)
```

```
function check_p2(A, x, G, g, h = 1e-4)

    # Pick a random direction
    δA = rand(size(A)...)
    δx = rand(size(x)...)

    # Compute an estimated directional derivative by finite differences
    np = norm((A+h*δA)*(x+h*δx))^2
    nm = norm((A-h*δA)*(x-h*δx))^2
    dn_fd = (np-nm)/(2*h)

    # Compute the analytical directional derivative
    dn_analytical = dot(g, δx) + dot(G, δA)

    # Return the relative error between the two computations
    abs((dn_fd-dn_analytical)/dn_fd)
end
```

```
A = 5×3 Matrix{Float64}:
 0.588466   0.741787  0.778003
 0.0170586  0.528736  0.0184582
 0.454404   0.242733  0.640032
 0.319548   0.537826  0.50365
 0.184154   0.930133  0.307337
```

```
A = rand(5,3)
```

```
x =  [0.139237, 0.176585, 0.332109]
```

```
x = rand(3)
```

```
G = 5×3 Matrix{Float64}:
 0.131246   0.166452   0.313051
 0.0283688  0.0359784  0.0676656
 0.0887476  0.112553   0.211682
 0.0854167  0.108329   0.203737
 0.0813026  0.103111   0.193924
```

```
G = 2*(A*x)*transpose(x)
```

```
g =  [1.15136, 1.83472, 1.6335]
```

```
g = 2*transpose(A) * A*x
```

```
error = 2.5521899789810134e-8
```

```
error = check_p2(A, x, G, g)
```

```
println(error)
```

```
2.5521899789810134e-8
```

3: Norm!

For $A = xy^T$, verify the following

$$\begin{aligned}\|A\|_1 &= \|x\|_1\|y\|_\infty \\ \|A\|_\infty &= \|x\|_\infty\|y\|_1 \\ \|A\|_F &= \|x\|_2\|y\|_2 \\ \|A\|_2 &= \|x\|_2\|y\|_2\end{aligned}$$

The 1-norm is the maximum column sum:
$\|A\|_1 = \max_j \sum_{i=1}^n |a_{ij}| = \max_j \sum_{i=1}^n |x_i||y_j| = \max_j |y_j| \cdot \|x\|_1 = \|y\|_\infty\|x\|_1 = \|x\|_1\|y\|_\infty$

The infinite-norm is the maximum row sum:
$\|A\|_\infty = \max_i \sum_{j=1}^n |a_{ij}| = \max_i \sum_{j=1}^n |x_i||y_j| = \max_i |x_i| \cdot \|y\|_1 = \|x\|_\infty\|y\|_1$

The Frobenius norm:
$\|A\|_F^2 = \sum_{i=1}^n \sum_{j=1}^n a_{ij}^2 = \sum_{i=1}^n \sum_{j=1}^n (x_i y_j)^2 = \sum_{i=1}^n x_i^2 \sum_{j=1}^n y_j^2 = \|x\|_2^2\|y\|_2^2$. Thus,
$\|A\|_F = \|x\|_2\|y\|_2$.

The 2-norm: For $A = xy^T$, the rank is 1, and the non-zero singular value is $\|A\|_2 = \|x\|_2\|y\|_2$ from SVD, where $A = \sigma uv^T$, and $\sigma = \|x\|_2\|y\|_2$.

4: Seeking structure

Suppose $A = D + uv^T$ where $D = \mathbf{diag}(d)$ for $d, u, v \in \mathbb{R}^n$. Rewrite each of the computations in the following code to take $O(n)$ time:

```
p4ref (generic function with 1 method)
```

```
# Reference computations
function p4ref(d, u, v, x)
    D = Diagonal(d)
    A = D + u*v'
    y1 = A*x      # Part 1
    y2 = A'*x     # Part 2
    d = diag(A)   # Part 3
    t = tr(A)     # Part 4
    y1, y2, d, t
end
```

```
p4mine (generic function with 1 method)
```

```
# Your computations (rewrite this)
function p4mine(d, u, v, x)
    vT_x = dot(v, x)
    uT_x = dot(u,x)
    vT_u = dot(v, u)
    y1 = Diagonal(d) * x + u * vT_x
    y2 = Diagonal(d) * x + v * uT_x
    dm = d + Diagonal(u) * v
    t = sum(d) + vT_u
    y1, y2, dm, t
end
```

```
let
    n = 1000     # Change this to 10000 as a sanity check!
    d = rand(n)
    u = rand(n)
    v = rand(n)
    x = rand(n)
    y1r, y2r, dr, tr = p4ref(d, u, v, x)
    y1m, y2m, dm, tm = p4mine(d, u, v, x)
    println(y1r ≈ y1m, y2r ≈ y2m, dr ≈ dm, tr ≈ tm)
end
```

```
truetruetruetrue
```